

DOC-08

VIXOR ENGINEERING BIBLE

كتاب قاعدة البيانات

الوثيقة الرسمية لهيكل وإدارة قاعدة بيانات فيكسور

جدول المحتويات

نظرة عامة على قاعدة البيانات (Database Overview)

جداول التداول (Trading Tables)

جداول المستخدم (User Tables)

جداول المحفظة والنقاط (Wallet & Points Tables)

جداول الذكاء الاصطناعي (AI Tables)

جداول المراقبة (Watching Tables)

جداول التجارب (Experiment Tables)

جداول النظام (System Tables)

العلاقات (Relations)

الفهارس (Indexes)

القيود (Constraints)

المشغلات (Triggers)

طرق العرض (Views)

الدوال (Functions)

سياسات الأمان (RLS Policies)

التحسين (Optimization)

معايير التسمية (Naming Standards)

قواعد الترحيل (Migration Rules)

استراتيجية النسخ الاحتياطي (Backup Strategy)

نظرة عامة على قاعدة البيانات (Database Overview)

تعتمد منظومة فيكسور على **Supabase** كمنصة قاعدة بيانات رئيسية، وهي بنية مبنية فوق **PostgreSQL** — أقوى أنظمة العلاقات مفتوحة المصدر. تُدار قاعدة البيانات عبر **Supabase Cloud** مما يوفر طبقة API مدمجة (PostgREST)، نظام مصادقة (Auth)، واشتراكات في الوقت الحقيقي (Realtime Subscriptions). يتم التفاعل مع قاعدة البيانات من جانب **supabase-js**، ومن جانب الخادم عبر **@supabase/ssr** مع دعم **Server-Side Rendering**.

تتكون قاعدة بيانات فيكسور من **27 جدولاً رئيسياً** مُقسّمة إلى سبع مجالات وظيفية: جداول التداول الخمسة (**analyses** ، **daily_signals** ، **signal_tracking** ، **price_alerts**)، وجداول المستخدم الستة (**profiles** ، **user_strategies** ، **user_streaks** ، **moxi_personas** ، **user_memories**)، وجداول النقاط والخطط (**credit_points** ، **points_balances** ، **points_transactions** ، **point_packs**)، وجداول الذكاء الاصطناعي الستة (**premium_subscriptions** ، **payments**)، وجداول المراقبة والتجارب والنظام. بالإضافة إلى جداول المراقبة والتجارب والنظام.

عدد الجداول	المنصة
27 جدولاً مُنظماً عبر 7 مجالات وظيفية رئيسية	PostgreSQL 15 — Supabase Cloud مع PostgREST API و Realtime Engine
الأمان	التعدادات
Row Level Security (RLS) على جميع الجداول مع <code>requireSupabaseAuth</code> كوسيط	4 تعدادات: <code>recommendation_type</code> ، <code>analysis_status</code> ، <code>signal_status</code> ، <code>points_reason</code>

✓ ميزة البنية التحتية

جميع أنواع TypeScript تولّد تلقائياً من مخطط قاعدة البيانات عبر `supabase gen types` وتُخزّن في `src/shared/supabase/types.ts` (1654 سطراً)، مما يضمن مزامنة دائمة بين المخطط والكود.

يستخدم نظام المصادقة في فيكسور نموذج **JWT-based** حيث يُمرّر رمز الوصول (Access Token) عبر `fireSupabaseAuth` عميلين: عميل التحقق (Validation Client) باستخدام مفتاح الخدمة (Service Role Key) للتحقق من صلاحية الرمز، وعميل (User JWT Client) باستخدام رمز المستخدم لضمان تطبيق سياسات **Row Level Security**. هذا النمط يضمن أن كل مستخدم الوصول إلا إلى بياناته الخاصة.

جداول التداول (Trading Tables)

تشكل جداول التداول القلب النابض لمنظومة فيكسور، حيث تُخزّن جميع البيانات المتعلقة بتحليلات السوق والصفقات والإشارات السعرية. تم تصميم هذه الجداول لتدعم دورة حياة كاملة للتداول بدءاً من التحليل التقني مروراً بإنشاء الإشارة وحتى تنفيذ الصفقة. جدول يخدم مرحلة محددة في هذه الدورة ويرتبط بالجدول الأخرى عبر مفاتيح أجنبية تضمن سلامة البيانات.

جدول التحليلات — analyses

الجدول الأكبر والأكثر تعقيداً في المنظومة. يخزن كل تحليل تقني يتم إجراؤه عبر الذكاء الاصطناعي أو يدوياً. يحتوي على 25 حقلاً (**recommendation**) من نوع **recommendation_type**، مستوى الثقة (**confidence**)، نقاط الدخول والخروج

stop_loss ، take_profit[] ، والهيكل السعري (market_structure) ككائن JSON. يمر كل تحليل عبر أربع مراحل: processing ← complete | failed .

trades — جدول الصفقات

يسجّل كل صفقة فعلية ينفذها المستخدم مع جميع تفاصيلها: الاتجاه (direction : long | short)، الحالة (status : cancelled)، أسعار الدخول والخروج، الكمية، الربح والخسارة (pnl ، pnl_pips)، ونسبة المخاطرة للعائد (r_multiple).
صفقة بتحليل محدد عبر analysis_id ومرتبطة بالمستخدم عبر user_id .

daily_signals — جدول الإشارات اليومية

جدول عام (Global) يحتوي على إشارات التداول اليومية لجميع أزواج العملات. لا يرتبط بمستخدم محدد بل يُولّد بواسطة النظام. يحتوي على زوج العملات، الإطار الزمني، التوصية، الثقة، ونقاط الدخول والخروج مع مناطق السيولة والهيكل السعري. يتم تتبع signal_tracking .

signal_tracking — جدول تتبع الإشارات

يربط الإشارات اليومية بالمستخدمين ويتتبع أداء كل إشارة. الحالة تتبع نوع signal_status : pending → active → cancelled / expired / sl_hit / tp3_hit / tp2_hit . يحتوي على بيانات حركة السعر (price : previous_price)، والانحراف الأقصى للأداء (max_adverse_excursion ، max_favorable_excursion)، وعدد المُحققة (hit_tp) .

price_alerts — جدول التنبيهات السعرية

يسمح للمستخدمين بإنشاء تنبيهات على أسعار محددة مع شروط مختلفة: above (أعلى من)، below (أدنى من)، crosses_down (يتجاوز صعوداً)، كل تنبيه مرتبط بالمستخدم ويحتوي على حالة (active ، cancelled) وتاريخ التفعيل.

الجدول	عدد الأعمدة	المفتاح الأساسي	المفاتيح الأجنبية	الوصف
analyses	25	id	user_id → profiles	تحليلات الذكاء الاصطناعي
trades	19	id	analysis_id ، user_id	سجل الصفقات
daily_signals	16	id	بدون	إشارات يومية عامة
signal_tracking	18	id	signal_id ، user_id	تتبع أداء الإشارات
price_alerts	11	id	profiles → user_id	تنبيهات الأسعار

03

جداول المستخدم (User Tables)

تدير جداول المستخدم البيانات الشخصية والتخصيصية لكل مستخدم مسجّل في المنظومة. تتبع فيكسور نهج Table Pattern . يتم تخزين البيانات الأساسية في جدول profiles المرتبط مباشرة بنظام المصادقة Supabase Auth عبر auth.users . تممّد هذه البنية لتشمل الإعدادات والاستراتيجيات والسلاسل والذاكرة والشخصيات.

profiles — جدول الملفات الشخصية

الجدول المركزي للمستخدم يحتوي على 13 حقلاً: المعرّف (id) الذي يطابق auth.uid() ، اسم العرض (display_name) المستخدم (username)، رابط الصورة الرمزية (avatar_url)، كود الإحالة (referral_code) المُولّد (gen_referral_code())، المُحيل (referred_by)، وأيام السلسلة (streak_days) بالإضافة إلى تكامل

telegram_id ، telegram_username ، telegram_photo_url). يحتوي على مفتاح أجنبي ذاتي (ed_by profiles.id) لتمثيل شجرة الحالات.

جدول الإعدادات — user_settings

علاقة واحد لواحد (1:1) مع المستخدم. يحتوي على إعدادات التطبيق المتقدمة: قنوات الإشعارات (notification_channels) مزود LLM المفضل (preferred_llm_provider)، مفاتيح API (llm_api_keys) المشفرة كـ JSON، بيانات exchange_credentials)، وإعدادات Telegram وWebhook. لا يمكن أن يكون للمستخدم أكثر من سجل إعدادات واحد.

جدول الاستراتيجيات — user_strategies

يسمح لكل مستخدم بتعريف استراتيجيات تداول متعددة. كل استراتيجية تحتوي على: اسم الاستراتيجية، أزواج العملات (pairs[])، نمط التداول (trading_style)، تحمل المخاطر (risk_tolerance)، والأطر الزمنية (preferred_timeframes[]). يمكن تفعيل استراتيجية واحدة فقط في كل لحظة عبر is_active.

جدول السلاسل — user_streaks

يتتبع سلسلة الأيام المتتالية التي يستخدم فيها المستخدم التطبيق. يحتوي على السلسلة الحالية (current_streak)، أو longest_streak)، وتاريخ آخر يوم مكتمل (last_completed_date). يُستخدم في نظام التحفيز ومكافأة النقاط عند تلبية شروط يومية.

جدول الشخصيات — moxi_personas

جزء من منظومة MOXI (Modular Orchestrated eXchange Intelligence). يخزن الشخصيات المخصصة التي يمكن للمستخدم أو تدريبها. كل شخصية تمثل أسلوب تحليل وتواصل مختلف (مثلاً: المحلل التقني المحافظ، المتداول العدواني، مدرب المخاطر). على طريقة توليد التحليلات والتفاعل عبر Copilot.

جدول الذاكرة — user_memories

نظام الذاكرة الدائم في PostgreSQL الذي يُمكن فيكسور من "تذكر" تفضيلات المستخدم وأنماط سلوكه وأخطائه ورؤاه واستراتيجيات تحتوي على فئة (category): (preference, behavior, mistake, insight, strategy)، محتوى النص (metadata). يُستخدم بواسطة MemoryStore لتخصيص تجربة المستخدم بشكل مستمر.

⚠️ خصوصية الذاكرة

بيانات الذاكرة حساسة لأنها تعكس أنماط سلوك المستخدم. يجب تطبيق سياسات RLS صارمة لضمان عدم وصول أي مستخدم لذاكرة مستخدم آخر. يتم إنشاء الذاكرة تلقائياً من التفاعلات ولا يمكن للمستخدم حذفها مباشرة.

04

جداول المحفظة والنقاط (Wallet & Points Tables)

تُشكّل جداول النقاط والاشتراكات منظومة اقتصادية متكاملة داخل فيكسور تسمح بإدارة الائتمانات والمشتريات والاشتراكات. تصميم هذه المنظومة وفق نمط Double-Entry Accounting المبسّط، حيث يتم تسجيل كل عملية في جدول المعاملات الرصيد تلقائياً عبر الدوال المخزّنة.

جدول أرصدة النقاط — points_balances

علاقة واحد لواحد مع المستخدم (المفتاح الأساسي هو user_id). يحتوي على رصيد النقاط الحالي (balance)، وإجمالي النقاط مدى الحياة (lifetime_earned)، وتاريخ آخر تحديث. تُعدّل هذه القيم حصرياً عبر الدوال المخزّنة (points() spend_points()) لضمان اتساق البيانات ومنع التعديل المباشر.

جدول معاملات النقاط — points_transactions

سجل دائم لكل حركة نقاط (Credit/Debit). كل معاملة تحتوي على: المبلغ (`delta`) يمكن أن يكون سالباً أو موجباً، السبب (`reason`)، عدد (8 أسباب: `points_reason` ، `signup_bonus` ، `analysis_cost` ، `pack_purchase` ، `bonus` ، `telegram_stars_purchase` ، `admin_adjust` ، `premium_grant` ، `daily_streak`)، وبيانات وصفية (`data`) هذا الجدول تتبعاً كاملاً لكل نقطة دخلت أو خرجت من حساب المستخدم.

جدول حزم النقاط — `point_packs`

تعريف الحزم المتاحة للشراء. يحتوي على: اسم الحزمة، عدد النقاط (`points`)، نقاط المكافأة الإضافية (`bonus_points`)، السعر (`price_cents`)، وشارة العرض (`badge`). يتم التحكم بالعرض عبر `is_active` و `sort_order` . لا يرتبط هذا الجدول بمحدد بل هو بيانات مرجعية عامة.

جدول خطط Premium — `premium_plans`

مشابه لجدول الحزم لكنه يُعرّف خطط الاشتراك المميزة. يحتوي على: اسم الخطة، الميزات (`features`) ك JSON، الفترة (`interval`)، السعر، وشارة العرض. تتحكم القيمة `is_active` بإظهار الخطة للمستخدمين.

جدول الاشتراكات — `premium_subscriptions`

يسجّل اشتراكات المستخدمين الفعلية في خطط Premium. يحتوي على: حالة الاشتراك (`status`)، تاريخ انتهاء الاشتراك (`current_period_end`)، والمفتاح الأجنبي `plan_id` المرتبط بـ `premium_plans` . يسمح لكل مستخدم باشتراك نشط واحد.

جدول المدفوعات — `payments`

سجل شامل لكل عملية شراء سواء كانت حزم نقاط أو اشتراكات مميزة. يدعم الدفع عبر Telegram Stars (`charge_id`)، المبلغ (`amount_stars`) وأنظمة أخرى. كل دفعة تحتوي على حمولة (`payload`)، حالة (`status`)، تاريخ التأكيد (`confirmed_at`)، الخطة أو الحزمة المشتراة عبر `plan_id` و `pack_id` .

الجدول	العلاقة بالمستخدم	المفاتيح الأجنبية	الوظيفة
<code>points_balances</code>	1:1	المفتاح = <code>user_id</code>	رصيد النقاط الحالي
<code>points_transactions</code>	N:1	<code>user_id</code>	سجل حركات النقاط
<code>point_packs</code>	بدون (مرجعي)	بدون	تعريف حزم الشراء
<code>premium_plans</code>	بدون (مرجعي)	بدون	تعريف خطط Premium
<code>premium_subscriptions</code>	N:1	<code>plans</code> → <code>plan_id</code> ، <code>user_id</code>	اشتراكات المستخدمين
<code>payments</code>	N:1	<code>plan_id</code> ، <code>pack_id</code> ، <code>user_id</code>	سجل المدفوعات

جداول الذكاء الاصطناعي (AI Tables)

تُمثّل جداول الذكاء الاصطناعي البنية التحتية لجميع قدرات فيكسور الذكية، من مساعد Copilot التفاعلي إلى نظام اتخاذ القرار (Agents). تتعامل هذه الجداول مع المحادثات والرسائل والأحداث Domain والقرارات والعمليات المدققة. التصميم يفصل المحادثة وبيانات العمليات لضمان الأداء والقابلية للتوسع.

جدول محادثات Copilot — `copilot_conversations`

يمثّل كل محادثة بين المستخدم ونظام Copilot. يحتوي على: عنوان المحادثة (`title`)، معرفّ الوكيل (`agent_id`) إذا كان متخصصاً، وعلم الإجماع (`is_consensus`) الذي يشير إلى ما إذا كانت النتيجة مبنية على توافق عدة وكلاء. كل محادثة مرتبطة بـ واحد عبر `user_id` .

يخزن كل رسالة (سؤال أو إجابة) ضمن محادثة. يحتوي على: الدور (role : user/assistant/system)، المحتوى (content)، الذي أجاب (agent_id)، وبيانات وصفية (metadata). مرتبط بـ copilot_conversations عبر conversation_id. هذا التصميم بمحادثات متعددة مع وكلاء مختلفين ضمن نفس المحادثة.

domain_events — جدول أحداث المجال

سجل أحداث مجال النظام (Domain Event Log). يُستخدم من طرف VixorEvents لتسجيل الأحداث الهامة مثل: إنشاء صفقة، إشعار مستخدم. يحتوي على: نوع الحدث (event_type)، حمولة البيانات (payload) كـ JSON، ويعتمد حقل الاختياري لربط الحدث بمستخدم محدد. يدعم Supabase Realtime للاشتراك في الأحداث الجديدة فوراً.

vixor_decisions — جدول قرارات فيكسور

يسجل كل قرار يتخذه نظام الذكاء الاصطناعي مع سياقه الكامل. يحتوي على: نوع القرار (decision_type)، العنوان (title)، التفصيلي (description)، بيانات القرار (data)، مستوى الثقة (confidence)، وملاحظات المستخدم (feedback). بيانات العملات الرقمية (token_symbol، chain، workspace، severity) لقرارات التداول.

agent_audit_log — جدول سجل تدقيق الوكلاء

سجل أمني شامل لكل طلب يُنفذه وكيل (Agent). يسجل: المسار (route)، الطريقة (method)، حالة الاستجابة (status)، التنفيذ بالمللي ثانية (duration_ms). يُستخدم للمراجعة الأمنية واكتشاف الأنماط المشبوهة وتحليل أداء الوكلاء.

agent_jobs — جدول عمليات الوكلاء

يتتبع العمليات غير المتزامنة التي ينفذها الوكلاء. كل عملية تحتوي على: الحالة (status)، نسبة التقدم (progress)، النتيجة كـ JSON، ورسالة الخطأ إن وجدت. مرتبط بالمستخدم والرمز المميز عبر user_id و token_id.

agent_tokens — جدول رموز الوكلاء

يدير رموز المصادقة (API Tokens) التي يستخدمها الوكلاء للوصول إلى المنظومة. كل رمز يحتوي على: تجميع SHA (hash)، نطاقات الصلاحيات (scopes[])، اسم وصفيًا، تاريخ انتهاء الصلاحية (expires_at)، وآخر استخدام. هذا يضمن وصول الو بأسلوب OAuth2 مبسط.

06

جداول المراقبة (Watching Tables)

تخدم جداول المراقبة وظيفة تتبع الأزواج المفضلة وإدارة ملاحظات التداول. تتيح هذه الجداول للمستخدمين تنظيم أسواقهم المفضلة الأزواج حسب استراتيجياتهم، وتدوين الملاحظات التي تربط بين التحليل والتنفيذ. التصميم يدعم قوائم مراقبة متعددة لكل مستخدم مخصص.

watchlists — جدول قوائم المراقبة

يمكن لكل مستخدم إنشاء قوائم مراقبة متعددة لتنظيم أزواج العملات. يحتوي على: اسم القائمة (name)، علم القائمة (is_default) الذي يحدد ما إذا كانت هذه هي القائمة التي تظهر عند فتح التطبيق، وترتيب العرض (sort_order). عند جديد، يُنشأ تلقائيًا قائمة مراقبة افتراضية واحدة. مرتبط بجدول profiles عبر user_id.

watchlist_items — جدول عناصر المراقبة

يخزن كل زوج عملات ضمن قائمة مراقبة محددة. يحتوي على: زوج العملات (pair)، التصنيف (category) الذي يمكن أن يكون "Major" أو "Crypto" أو "Exotic"، ملاحظات مخصصة (notes)، وترتيب العرض (sort_order). مرتبط بـ watchlists. يمكن لنفس الزوج أن يظهر في قوائم مراقبة متعددة لنفس المستخدم أو لمستخدمين مختلفين.

جدول ملاحظات التداول — trading_notes

دفتر يومية التداول الرقمي للمستخدم. يمكن ربط كل ملاحظة بزوج عملات محدد (pair) أو بتحليل محدد (analysis_id) يحتوي على: العنوان (title)، المحتوى (content)، وسوم (tags[])، المزاج (mood : cautious | anxious | neutral)، وعلم التثبيت (is_pinned). يدعم هذا الجدول ميزة Trading Journal المهمة لتحسين أداء المستخدم.

ملاحظات التداول

دفتر يومية رقمي مع مزاج ووسوم وتثبيت وربط بالتحليلات

قوائم مراقبة

تنظيم الأزواج في مجموعات مخصصة مع تصنيف وترتيب مرن

إلغاء متتالي

حذف قائمة مراقبة يلغي تلقائياً جميع عناصرها عبر SCAD

علاقة تسلسلية

profiles → watchlists → watchlist_items (1:N:N)

07

جداول التجارب (Experiment Tables)

تُمكن جداول التجارب منظومة فيكسور من تنفيذ تجارب خوارزمية للتحسين المستمر لاستراتيجيات التداول. هذه الجداول تنتمي لـ **ENGINES** (محرك التجارب) وتدعم نمط **Generational Optimization** حيث تُقَيَّم كل جيل من الحلول وتُحسَّن عبر الخوارزميات. التصميم يفصل بين إعدادات التجربة وتوليداته لضمان المرونة وسهولة التحليل.

جدول التجارب — experiments

يمثّل كل تجربة مستقلة يُنفذها المستخدم أو النظام تلقائياً. يحتوي على: إعدادات التجربة (config) ككائن JSON يحتوي على الخوارزمية والقيود، نتيجة التجربة النهائية (result) كـ JSON، الحالة (status): running | completed | failed، تاريخ مرتبط بالمستخدم عبر user_id . يمكن للمستخدم إجراء تجارب متعددة بالتوازي.

جدول توليدات التجربة — experiment_generations

يخزن بيانات كل جيل (Generation) ضمن التجربة. في الخوارزميات الجينية، كل جيل يمثل مجموعة من الحلول المُقَيَّمة. يحتفظ الجيل (generation)، أفضل نتيجة (best_score)، متوسط النتائج (avg_score)، والمجموعة الكاملة من الحلول (solutions). مرتبط بـ experiments عبر experiment_id . هذا الجدول يسمح برسم منحنى التحسين عبر الأجيال وتحليل تقارب الحلول.

✓ مرونة التصميم

استخدام JSON لتخزين config وresult وpopulation يُمكن من دعم أنواع مختلفة من التجارب دون تعديل مخطط قاعدة البيانات. إضافة معلمات جديدة أو تغيير بنية الحلول بمرونة كاملة.

مثال: هيكل تجربة خوارزمية

```
g": {
  "algorithm": "genetic",
  "population_size": 100,
  "mutation_rate": 0.05,
  "crossover_rate": 0.8,
  "max_generations": 50,
  "fitness_function": "sharpe_ratio",
  "constraints": { "max_risk": 0.02 }
},
"best_solution": { "params": {...}, "sharpe": 2.1 },
"total_generations": 42,
"convergence": true
```

08

جداول النظام (System Tables)

جدول الإشعارات (`notifications`) هو الجدول الرئيسي في مجال النظام ويُمثّل البنية التحتية للتواصل مع المستخدمين. الجدول يدعم قنوات إشعاع متعددة (In-App، Telegram، Email، Webhook) وأنواع مختلفة من الإشعارات. يحتوي على (العنوان، المحتوى، النوع، حالة القراءة) وحقل إضافية للتوسع (قناة الإرسال، الحالة، حمولة البيانات، تاريخ الإرسال، رسالة الخطأ).

جدول الإشعارات — notifications

كل إشعار يحتوي على: عنوان (`title`)، نص المحتوى (`body`)، نوع الإشعار (`type`) مثل "signal" أو "trade" أو "alert". القراءة مع تاريخ القراءة (`read_at`). الحقول الممتدة تشمل: قناة الإرسال (`channel`)، حمولة البيانات (`payload`)، حالة الإرسال (`status` : pending | sent | failed)، وتاريخ الإرسال الفعلي (`sent_at`) مع رسالة الخطأ إن فشل.

دورة حياة الإشعار

تمر الإشعارات بعدة مراحل: الإنشاء → الإرسال عبر القنوات المفعّلة → تسجيل الحالة. عند نجاح الإرسال، يُحدّث (`sent_at`) ويُسجّل الخطأ في (`error`) ويمكن إعادة المحاولة. المستخدم يرى الإشعارات في التطبيق ويمكنه تمييزها كمقروءة عبر تحديث (`read_at`).

⚠️ صيانة الإشعارات

يجب تنفيذ عملية تنظيف دورية (Cleanup Job) لحذف الإشعارات القديمة المقروءة (أقدم من 30 يوماً مثلاً) لمنع تضخم الجدول. يمكن استخدام (`created_at`) كمعيار للحذف مع الاحتفاظ بالإشعارات غير المقروءة.

أنواع الإشعارات

signal, trade, alert, system, premium, streak

القنوات المدعومة

In-App, Telegram Bot, Email, Webhook

إعادة المحاولة

عند الفشل يُسجّل الخطأ ويمكن إعادة الإرسال تلقائياً

Payload مرّن

كائن JSON يحتوي على بيانات مخصصة حسب نوع الإشعار

العلاقات (Relations)

تعتمد قاعدة بيانات فيكسور على نظام علاقات مرتبط عبر المفاتيح الأجنبية (Foreign Keys) التي تربط الجداول ببعضها البعض. البيانات Referential Integrity. تُدار العلاقات عبر Supabase وتُعكّس تلقائياً في أنواع TypeScript المُولّدة. فهم هذه العلاقات لتصميم الاستعلامات الفعالة وتحليل البيانات عبر الجداول المتعددة.

العلاقة المركزية — profiles ↔ جميع الجداول

جدول `profiles` هو المركز الذي تتفرع منه جميع البيانات. ما يقارب 15 جدولاً تحتوي على `user_id` كمفتاح أجنبي. هذا يخلق علاقة **واحد لعدة (N:1)** من الملف الشخصي إلى كل جدول فرعي. التصميم يضمن حذف البيانات **CASCADE DELETE** عند حذف حساب مستخدم.

العلاقات التسلسلية

- **الإشارات:** `daily_signals` → `signal_tracking` (واحد لعدة — إشارة واحدة يمكن تتبعها من عدة مستخدمين)
- **المحادثات:** `copilot_conversations` → `copilot_messages` (واحد لعدة — محادثة تحتوي رسائل متعددة)
- **المراقبة:** `watchlists` → `watchlist_items` (واحد لعدة — قائمة تحتوي عناصر متعددة)
- **التجارب:** `experiments` → `experiment_generations` (واحد لعدة — تجربة تحتوي أجيال متعددة)
- **التحليلات:** `analyses` → `trades` و `analyses` → `trading_notes` (واحد لعدة)

العلاقات المرجعية (Lookup)

- **الاشتراكات:** `premium_subscriptions` → `premium_plans` (عدة لواحد — عدة اشتراكات تشير لنفس الخطة)
- **المدفوعات:** `payments` → `point_packs` و `payments` → `premium_plans` (عدة لواحد)

العلاقة الذاتية

جدول `profiles` يحتوي على علاقة ذاتية حيث يشير `referred_by` إلى `profiles.id` في نفس الجدول. هذا يمثّل متعددة المستويات حيث يمكن للمستخدم "أ" أن يشير إلى المستخدم "ب" الذي أشار إليه المستخدم "ج". يجب الحذر من التعمق الاستعلام عن هذه الشجرة.

العلاقات الاختيارية

بعض المفاتيح الأجنبية اختيارية (Nullable) لتوفير المرونة: `trades.analysis_id` (صفقة قد لا تكون مرتبطة بـ تحليل) و `signal_tracking.signal_id` (تتبع قد يكون يدوياً)، `vixor_decisions.expires_at` (قرار قد لا يكون له مدة المرونة ضرورية لدعم سيناريوهات الاستخدام المتنوعة).

```
(1) → (N) analyses
(1) → (N) trades
(1) → (N) signal_tracking
(1) → (N) price_alerts
(1) → (N) copilot_conversations
(1) → (N) user_strategies
(1) → (N) user_streaks
(1) → (1) user_settings
(1) → (1) points_balances
(1) → (N) points_transactions
(1) → (N) premium_subscriptions
(1) → (N) payments
(1) → (N) watchlists
(1) → (N) trading_notes
(1) → (N) experiments
(1) → (N) vixor_decisions
(1) → (N) user_memories
(1) → (N) agent_tokens

(1) → (N) trades
(1) → (N) trading_notes
signals (1) → (N) signal_tracking
conversations (1) → (N) copilot_messages
ts (1) → (N) watchlist_items
ts (1) → (N) experiment_generations
plans (1) → (N) premium_subscriptions
cks (1) → (N) payments
kens (1) → (N) agent_jobs
(1) → (1) profiles [referred_by]
```

10

الفهارس (Indexes)

الفهارس (Indexes) هي العنصر الأهم لتحسين أداء الاستعلامات في PostgreSQL. بدون فهارس مناسبة، يضطر محرك قاعدة إجراء مسح كامل للجدول (Sequential Scan) مما يُبطئ الاستعلامات بشكل كبير مع نمو حجم البيانات. في فيكسور، معظم تُفلتر حسب `user_id` بسبب سياسات RLS، مما يجعل فهارس هذا العمود الأولوية القصوى.

فهارس المستخدم (Critical — الأولوية القصوى)

بما أن سياسات RLS تُضيف شرط `WHERE user_id = auth.uid()` تلقائياً لكل استعلام، فإن جميع الجداول التي `user_id` تحتاج فهرساً على هذا العمود. بدون هذا الفهرس، سيُحوّل كل استعلام بسيط إلى مسح كامل للجدول. يُنصح باستخدام **BRIN** للجدول الكبيرة المرتبة زمنياً و**B-tree** للباقي.

الجدول	الأعمدة	النوع	السبب
analyses	(user_id, created_at DESC)	B-tree	RLS + ترتيب زمني
analyses	(status)	B-tree	فلتر حسب الحالة
trades	(user_id, status)	B-tree	RLS + فلتر الصفقات الم
trades	(user_id, pair)	B-tree	استعلامات حسب الزوج
signal_tracking	(user_id, status)	B-tree	RLS + فلتر الإشارات النش
daily_signals	(pair, signal_date)	B-tree	استعلامات الإشارات اليوم
copilot_messages	(conversation_id, created_at)	B-tree	ترتيب رسائل المحادثة
notifications	(user_id, read_at)	B-tree	RLS + فلتر المقروءة
points_transactions	(user_id, created_at DESC)	B-tree	سجل المعاملات بالترتيب
domain_events	(event_type, created_at)	B-tree	استعلامات حسب نوع الح
price_alerts	(user_id, status)	B-tree	RLS + فلتر التنبيهات النش
agent_audit_log	(user_id, created_at)	BRIN	سجل كبير مرتب زمنياً

⚠ تحذير الأداء

جدول agent_audit_log وdomain_events ينموان بسرعة كبيرة. بدون فهارس مناسبة وأرشفة دورية، سيتدهور الأداء بشكل ملحوظ. **BRIN** للجدول التي تُضاف إليها بيانات مرتبة زمنياً لأنه أصغر وأكثر كفاءة من B-tree في هذه الحالة.

القيود (Constraints)

القيود (Constraints) في PostgreSQL تُنفّذ قواعد البيانات على مستوى المخطط لضمان صحة واتساق البيانات. تمنع القيود غير صالحة أو غير متسقة وتضمن سلامة البيانات حتى في حالة وجود أخطاء برمجية في طبقة التطبيق. في فيكسور، تُستخدم رئيسية من القيود: CHECK، UNIQUE، NOT NULL، وFOREIGN KEY.

قيود NOT NULL

تضمن أن حقولاً محددة لا يمكن أن تحتوي على قيم فارغة. في فيكسور، الحقول التالية إلزامية: `user_id` في جميع الجداول، `status` في جداول الحالة مثل `analyses` و `signal_tracking` و `trades`، `pair` في جدول `trades`، `content` في `copilot_messages` و `user_memories`، `recommendation` في `analyses` (يمكن أن يكون NULL للفشل)، و `delta` و `reason` في `points_transactions`.

قيود UNIQUE

تضمن عدم تكرار القيم. في فيكسور: `profiles.referral_code` يجب أن يكون فريداً عالمياً، `profiles.settings.user_id` إعداداً واحداً لكل مستخدم (علاقة 1:1)، `points_balances.user_id` يضمن رصيداً واحداً لكل مستخدم. كود الإحالة يُنشأ باستخدام `gen_referral_code()` التي تضمن التفرد عبر التحقق من عدم وجود الكود مسبقاً.

قيود CHECK

تُنفَّذ قواعد التحقق على مستوى القيم. التعدادات (ENUM) تُمثَّل أحد أشكال قيود CHECK. في فيكسور: recommendation_type ,analysis_status ,BUY | SELL | WAIT ,analysis_status يقتصر على CHECK ,points_reason يقتصر على 8 حالات محددة، balance ≥ 0 , confidence ≥ 0 AND confidence ≤ 100 في points_balances .

قيود FOREIGN KEY

تضمن سلامة العلاقات بين الجداول. جميع المفاتيح الأجنبية في فيكسور تستخدم ON DELETE CASCADE أو SET NULL حسب السياق. CASCADE يُستخدم عندما يجب حذف البيانات الفرعية مع البيانات الرئيسية (مثل حذف قائمة مراقبة يحذف NULL يُستخدم عندما يجب الاحتفاظ بالبيانات مع إزالة المرجع (مثل حذف تحليل لا يحذف الصفقات المرتبطة به).

✓ مبدأ الدفاع في العمق

القيود على مستوى قاعدة البيانات هي خط الدفاع الأخير، حتى لو كانت طبقة التطبيق تحتوي على تحقق كامل، تظل القيود ضرورية لحماية من: أخطاء البرمجة، استعلامات مباشرة، وحالات التزامن المتزامن (Race Conditions).

12

المشغلات (Triggers)

المشغلات (Triggers) في PostgreSQL هي إجراءات تُنفَّذ تلقائياً عند حدوث أحداث محددة على جدول (UPDATE، DELETE، INSERT). تُستخدم في فيكسور لإدارة القيم الافتراضية، تسجيل التغييرات، وتنفيذ منطق العمل على مستوى قاعدة البيانات. تصميم المشغلات يُقلل من التعقيد في طبقة التطبيق ويضمن تنفيذ المنطق بشكل متسق.

مشغلات الطوابع الزمنية (Auto-Timestamps)

جميع الجداول في فيكسور تحتوي على created_at و updated_at . يُعيّن created_at تلقائياً عبر قيمة افتراضية الإيداع، بينما يُحدَّث updated_at تلقائياً عبر مشغّل BEFORE UPDATE . هذا يضمن دقة الطوابع الزمنية دون اعتماد على طابع الوقت.

مشغلات حذف متتالي (CASCADE Deletes)

عند حذف ملف شخصي (Profile)، تُحذف تلقائياً جميع البيانات المرتبطة عبر المفاتيح الأجنبية مع ON DELETE CASCADE . هذا يضمن الحذف الشامل للبيانات الفرعية. يجب الحذف هذا لأن الحذف لا يمكن التراجع عنه (Hard Delete).

مشغلات توليد كود الإحالة

عند إنشاء ملف شخصي جديد، يُنشأ كود إحالة فريد تلقائياً عبر الدالة gen_referral_code() . يمكن تنفيذ هذا المشغّل BEFORE INSERT على جدول profiles . يُولّد الكود إذا لم يتم توفيره. الدالة تُنتج كوداً عشوائياً وتتحقق من عدم تكراره في الجدول قبل الإدراج.

مشغلات مخصصة مقترحة

- تحديث السلسلة: مشغّل AFTER UPDATE على user_streaks يُرسل إشعاراً عند تجاوز سلسلة جديدة أطول سلسلة سابقة.
- تنبيهات الأسعار: مشغّل يعمل بشكل دوري عبر pg_cron لفحص price_alerts النشطة ومقارنتها بالأسعار الحالية.
- انتهاء الإشارات: مشغّل يُحدَّث حالة الإشارات المنتهية الصلاحية من active إلى expired .
- تنظيف الإشعارات: مشغّل مجدول عبر pg_cron يحذف الإشعارات المقروءة الأقدم من 30 يوماً.
- تنظيف سجل التدقيق: مشغّل يحذف سجلات agent_audit_log الأقدم من 90 يوماً.

```

-- مثال: مشغل تحديث الطابع
CREATE TRIGGER update_updated_at()
    AFTER UPDATE ON analyses
    BEGIN
        UPDATE analyses SET updated_at = CURRENT_TIMESTAMP
        WHERE id = NEW.id;
    END;

```

طرق العرض (Views)

طرق العرض (Views) في PostgreSQL توفر طبقة تجريد فوق الجداول الأساسية، تسمح بتبسيط الاستعلامات وتحسين الأداء عبر تجميع البيانات مسبقاً، وتوفير واجهات مُعاد تسميتها لواجهة برمجة التطبيقات. في فيكسور، تُدعى العرض المقترحة لتقديم رؤى جاهزة وملخصات دون الحاجة إلى استعلامات معقدة في كل مرة.

عرض إحصائيات المستخدم — user_stats

طريقة عرض تجمّع بيانات المستخدم من عدة جداول في صف واحد. تحتوي على: إجمالي الصفقات، معدل الربح (إجمالي R-Multiple، متوسط الثقة في التحليلات، رصيد النقاط، طول السلسلة الحالية، وعدد الملاحظات. يُبسّط التحكم (Dashboard) التي تحتاج بيانات من عدة جداول.

عرض أداء الإشارات — `signal_performance`

تربط بين `daily_signals` و `signal_tracking` لحساب أداء كل إشارة: نسبة النجاح، متوسط R-Multiple، ضرب كل TP. يُستخدم في تقارير الأداء وفي صفحة الإحصائيات لمساعدة المستخدمين على تقييم جودة الإشارات.

عرض المحفظة — user_wallet_summary

ندمج بين `points_balances` ، `premium_subscriptions` ، وآخر معاملة من `points_transactions` لتقديم
على المحفظة: الرصيد الحالي، حالة الاشتراك، وتاريخ آخر عملية.

عرض سجل النشاط — user_activity_feed

نشاط واحد مرتب زمنياً. كل سجل يحتوى على: نوع النشاط، الوصف، التاريخ، والبيانات المرتبطة.

active_analyses — عرض التحليلات النشطة

تعرض التحليلات المكتملة (`status = 'complete'`) التي لم تنتهِ صلاحيتها بعد مع إرفاق بيانات `signal_tracking` إذا وُجدت. تُستخدم في لوحة التحكم الرئيسية لعرض التحليلات الحالية ذات الصلة.

مثال: طريقة عرض إحصائيات الدوال

```
SELECT user_stats AS
```

```
SELECT user_id,
       display_name,
       (SELECT COUNT(*) FROM trades t WHERE t.user_id = p.id) AS total_trades,
       (SELECT COUNT(*) FROM trades t WHERE t.user_id = p.id AND t.pnl > 0)::float /
       NULLIF((SELECT COUNT(*) FROM trades t WHERE t.user_id = p.id), 0) AS win_rate,
       balance AS points_balance,
       lifetime_earned AS points_earned,
       current_streak,
       longest_streak,
       (SELECT COUNT(*) FROM analyses a WHERE a.user_id = p.id) AS total_analyses
FROM profiles p
LEFT JOIN points_balances pb ON pb.user_id = p.id
LEFT JOIN user_streaks us ON us.user_id = p.id;
```

14

الدوال (Functions)

الدوال المخزنة (Stored Functions) في PostgreSQL تُنفذ منطق العمل على مستوى قاعدة البيانات مما يُحسن الأداء وتنفيذ العمليات بشكل ذري (Atomic). في فيكسور، تُستخدم الدوال لإدارة النقاط، توليد الأكواد، وتنفيذ عمليات التساقص. التعامل مع النقاط يتم حصرياً عبر الدوال المخزنة لمنع أي تعديل مباشر للأرصدة.

دالة توليد كود الإحالة — `gen_referral_code()`

تُولد كود إحالة فريد من 8 أحرف. تعمل بدون معلمات وتُرجع سلسلة نصية. الخوارزمية تولّد كوداً عشوائياً من 0-9 وأحرف A-Z وتتأكد من عدم وجوده في `profiles.referral_code`. إذا وُجد الكود (احتمال ضئيل جداً)، تُعيد التوليد. تُستخدم لإنشاء ملف شخصي جديد.

دالة إضافة النقاط — `credit_points()`

تضيف نقاطاً لحساب المستخدم بشكل ذري. المعلمات: `_user` (UUID)، `_amount` (number)، `_meta` (JSON اختياري). تقوم بثلاث عمليات في معاملة واحدة: التحقق من وجود الرصيد، إضافة `balance` و `lifetime_earned` في `points_balances`، وتسجيل المعاملة في `points_transactions`. `credit_points()` تُرجع `points_earned` الجديد.

دالة صرف النقاط — `spend_points()`

تخصم نقاطاً من حساب المستخدم مع التحقق من `sufficiency`. المعلمات مماثلة لـ `credit_points()` لكن `spend_points()` يجب أن يكون موجّباً. تقوم بـ: التحقق من كفاية الرصيد، طرح المبلغ من `balance`، وتسجيل المعاملة. إذا كان الرصيد سلباً، تُرجع خطأ يمنع العملية بالكامل.

دوال مقترحة

- `transfer_points()`: نقل نقاط بين مستخدمين (للإحالات أو الهدايا) مع التحقق من الكفاية وتسجيل المعاملتين.
- `get_user_stats()`: إرجاع ملخص شامل لبيانات المستخدم من عدة جداول في استدعاء واحد.
- `expire_old_alerts()`: تعيين حالة التنبيهات المنتهية إلى `expired` بشكل مجمع.
- `cleanup_old_data()`: حذف البيانات القديمة مع حفظ إحصائيات مجملة.
- `recalculate_streak()`: إعادة حساب سلسلة المستخدم عند تغير التاريخ.

يُمنع منعاً باتاً تعديل `points_balances.balance` مباشرة عبر `UPDATE`. جميع عمليات النقاط يجب أن تمر عبر `credit_points()` أو `spend_points()` فقط. هذا يضمن تسجيل كل عملية في `points_transactions` ويمنع التناقض المالية.

سياسات الأمان (RLS Policies)

Row Level Security (RLS) هو آلية أمان في PostgreSQL تُقيّد الوصول على مستوى الصفوف (Rows) وليس الجداول فيكسور، تُفَعّل RLS على جميع الجداول بحيث لا يستطيع أي مستخدم الوصول إلا إلى بياناته الخاصة. تُضاف سياسات ترفض كل الوصول (`DENY ALL`) عند تفعيل RLS، ثم تُضاف سياسات محددة للسماح بالعمليات المطلوبة.

نمط السياسات في فيكسور

تتبع فيكسور نمط User-Isolation Policy حيث كل سياسة تحتوي على شرط `auth.uid() = user_id`. Supabase يُضيف تلقائياً هذا الشرط لكل استعلام يأتي من العميل. على سبيل المثال، عندما يطلب المستخدم جدول `analyses` يتحول الاستعلام تلقائياً إلى: `SELECT * FROM analyses WHERE user_id = 'user-uuid'`.

وسيط المصادقة — requireSupabaseAuth

نظام المصادقة في فيكسور يعمل عبر وسيط (Middleware) يُنفذ قبل كل دالة الخادم. الوسيط يستخرج رمز `Authorization Header`، ثم يُنشئ عميلين لـ Supabase: عميل التحقق باستخدام `Auth Key` للتحقق من صلاحية الرمز مع `auth.getUser(token)`، و عميل المستخدم باستخدام `User JWT` الذي يُفَعّل RLS فشل التحقق، يُرجع الوسيط خطأ `401 Unauthorized`.

أنواع السياسات المُطبّقة

نوع العملية	السياسة	الوصف
<code>SELECT</code>	قراءة البيانات الخاصة	المستخدم يقرأ صفوفه فقط
<code>INSERT</code>	إدراج بيانات مرتبطة	المستخدم يُدرج بـ <code>auth.uid() = user_id</code>
<code>UPDATE</code>	تحديث البيانات الخاصة	المستخدم يُعدّل صفوفه فقط
<code>DELETE</code>	حذف البيانات الخاصة	المستخدم يحذف صفوفه فقط

استثناءات RLS

بعض الجداول تحتاج استثناءات: `daily_signals` هو جدول عام (Global) يمكن لأي مستخدم قراءته لأنه ليست خاصة بمستخدم محدد. `point_packs` و `premium_plans` هما جداول مرجعية عامة. `in_events` قراءته من قبل أي مستخدم مصادق عليه. لهذه الجداول، تُستخدم سياسات `SELECT USING (true)` للسماح العامة مع الحفاظ على RLS للعمليات الأخرى.

⚠ تحذير أمني بالغ الأهمية

لا تستخدم أبداً مفتاح الخدمة (Service Role Key) في العميل (Browser). مفتاح الخدمة يتجاوز RLS تماماً ويمنح صلاحية على جميع البيانات. استخدمه فقط في جانب الخادم (Server Functions) مع الوسيط `requireSupabaseAuth`. في الخادم استخدم دائماً `getSupabaseOrThrow()` الذي يُرجع عميلاً بـ `JWT` المستخدم.


```

نموذجية RLS مثال:
GRANT USAGE ON SCHEMA analytics TO authenticated;
REVOKE ANALYZE ON SCHEMA analytics FROM authenticated;
GRANT SELECT ON analytics TO authenticated;
GRANT INSERT ON analytics TO authenticated;
GRANT UPDATE ON analytics TO authenticated;
GRANT DELETE ON analytics TO authenticated;

POLICY "users_read_own_analyses"
ON analytics FOR SELECT
USING (auth.uid() = user_id);

POLICY "users_insert_own_analyses"
ON analytics FOR INSERT
USING (auth.uid() = user_id);
CHECK (auth.uid() = user_id);

POLICY "users_update_own_analyses"
ON analytics FOR UPDATE
USING (auth.uid() = user_id)
CHECK (auth.uid() = user_id);

POLICY "users_delete_own_analyses"
ON analytics FOR DELETE
USING (auth.uid() = user_id);

استثناء: قراءة عامة للبيانات
GRANT USAGE ON SCHEMA analytics TO public;
REVOKE ANALYZE ON SCHEMA analytics FROM public;
GRANT SELECT ON analytics TO public;
GRANT INSERT ON analytics TO public;
GRANT UPDATE ON analytics TO public;
GRANT DELETE ON analytics TO public;

POLICY "authenticated_read"
ON analytics FOR SELECT
USING (auth.role() = 'authenticated');

```

16

التحسين (Optimization)

تحسين أداء قاعدة البيانات هو عملية مستمرة تتطلب مراقبة وتعديلاً دورياً. في فيكسور، مع نمو عدد المستخدمين وحجم البيانات، يصبح التحسين ضرورياً للحفاظ على سرعة الاستجابة. يتضمن التحسين عدة محاور: الاستعلامات، إدارة الاتصالات، تجميع البيانات، والتخزين المؤقت.

تحسين الاستعلامات (Query Optimization)

- استخدام **SELECT** محدد: تجنب **SELECT *** وحدد الأعمدة المطلوبة فقط لتقليل حجم البيانات المنقولة
- الترقيم (Pagination): استخدم **range(0, 19)** أو **limit(20)** لتحميل البيانات على دفعات
- الفلاتر المبكرة: طبق فلاتر **eq('status', 'active')** قبل الترتيب للاستفادة من الفهارس
- تفادي JOINS غير ضرورية: استخدم الاستعلامات المباشرة بدلاً من JOINS المعقدة عند الإمكان
- **EXPLAIN ANALYZE**: راجع خطة التنفيذ للاستعلامات البطيئة عبر لوحة تحكم Supabase

إدارة الاتصالات (Connection Pooling)

Supabase يوفر Connection Pooling الذي يدير مجموعة اتصالات مشتركة. في فيكسور، يُستخدم Transaction Mode عبر Port 6543 لعميل الخادم، حيث يُعطى كل معاملة اتصالاً من المجموعة. للعميل يُستخدم وضع Session Mode عبر Port 5432. هذا يُقلل عدد الاتصالات الفعلية إلى PostgreSQL ويمنع استنزاف الموارد.

التخزين المؤقت (Caching)

تُستخدم استراتيجيات تخزين مؤقت متعددة في فيكسور: Supabase Cache مدمج مع الاستعلامات (30 ثانية)، Browser Cache عبر TanStack Query مع تحديد **staleTime** و **gcTime**، و Local Memory Cache على الخادم. المراجعة مثل **premium_plans** و **point_packs** التي نادراً ما تتغير. البيانات التي لا تتغير أبداً (جداول مرجعية) تُخزن مؤقتاً لفترة قصيرة أو تُستخدم Subscriptions للتحديثات الفورية.

بدلاً من الاستقصاء الدوري (Polling)، تُستخدم Supabase Realtime للاستماع للتغييرات على الجداول التي يُقلل الحمل على قاعدة البيانات ويوفر تحديثات فورية. تُستخدم Realtime بشكل أساسي لـ: `analyses` (للتنبؤات)، `notifications` (للإشعارات الجديدة)، و `signal_tracking` (للتتبع الإشارات).

⚠ مراقبة الأداء

راقب مؤشرات الأداء عبر لوحة تحكم Supabase بانتظام: عدد الاتصالات النشطة، أطول الاستعلامات (Slow Queries) استهلاك التخزين، ومعدل القراءة/الكتابة. إذا تجاوز وقت الاستعلام 100 ميلي ثانية، فهو مرشح للتحسين.

17

معايير التسمية (Naming Standards)

معايير التسمية (Naming Conventions) في قاعدة بيانات فيكسور تتبع نظاماً صارماً ومتسقاً يُسهّل فهم والتعاون بين المطوّرين. الاختيار المتسق للأسماء يُقلل من الأخطاء البرمجية ويجعل قاعدة البيانات أكثر قابلية للتوسع. تتبع فيكسور معايير PostgreSQL و Supabase الموصى بها.

أسماء الجداول (Table Names)

- الصيغة: جمع باللغة الإنجليزية، أحرف صغيرة فقط، مفصولة بشرطة سفلية (`snake_case`)
- أمثلة: `analyses` ، `daily_signals` ، `user_strategies` ، `copilot_conversations`
- البادئات: `user_` للجداول المرتبطة بالمستخدم، `agent_` لجداول الوكلاء
- التركيب: عند وجود علاقة، يُسمّى الجدول الفرعي باسم مركب مثل `watchlist_items` (عناصر قائمة المراقبة)

أسماء الأعمدة (Column Names)

- الصيغة: `snake_case` ، أحرف صغيرة، وصفية ودقيقة
- المفاتيح الأساسية: `id` (UUID مُولّد عبر `gen_random_uuid()`)
- المفاتيح الأجنبية: `{table}_id` مثل `user_id` ، `analysis_id` ، `watchlist_id`
- الطوابع الزمنية: `created_at` ، `updated_at` ، `{event}_at` مثل `sent_at` ، `triggered_at`
- الأعلام (Booleans): `is_{adjective}` مثل `is_active` ، `is_default` ، `is_pinned`
- المصفوفات: جمع بالإنجليزية مع أقواس مثل `tags[]` ، `take_profit[]` ، `scopes[]`

أسماء التعدادات (Enum Names)

- الصيغة: `{entity}_{attribute}` بالـ `snake_case`
- أمثلة: `recommendation_type` ، `analysis_status` ، `signal_status` ، `points_reason`
- القيم: أحرف صغيرة أو `UPPER_CASE` حسب السياق (مثل `BUY|SELL|WAIT`)

أسماء الدوال والمشغلات

- الدوال: فعل اسم مثل `gen_referral_code` ، `credit_points` ، `spend_points`
- المشغلات: `{action}_{table}` مثل `set_updated_at`
- سياسات RLS: `{operation}_{subject}_{object}` مثل `users_read_own_analyses`

أنواع البيانات

PostgreSQL في	الاستخدام	الأمثلة في فيكسور
uuid	المفاتيح الأساسية والأجنبية	user_id ، id
varchar / text	النصوص القصيرة والمتوسطة	pair ، content ، title
jsonb	بيانات JSON المرنة	features ، config ، metadata
text[]	مصفوفات النصوص	scopes ، take_profit ، tags
numeric	الأرقام الدقيقة	price ، balance ، confidence
boolean	الأعلام المنطقية	is_default ، is_active
timestampz	التواريخ والأوقات	updated_at ، created_at
enum	القيم المحددة مسبقاً	signal_status ، recommendation_type

18

قواعد الترحيل (Migration Rules)

الترحيل (Migrations) في فيكسور تُدار عبر Supabase Migrations وهي مجموعة من ملفات SQL المُرقّمة بالتتابع لتعديل مخطط قاعدة البيانات. كل ترحيل يمثل تغييراً واحداً أو مجموعة تغييرات مترابطة على المخطط (Development، Staging، Production) لضمان تطابق المخطط عبر جميع البيئات.

صيغة أسماء الترحيلات

تتبع فيكسور صيغة `{timestamp}_{description}.sql` حيث الطابع الزمني هو رقم يونيكس. مثال: `20260612000000_add_domain_events.sql`. الترقيم يضمن التنفيذ بالترتيب الصحيح. الوصف يجب أن يكون واضحاً بالإنجليزية ويصف التغيير الرئيسي.

قواعد الترحيل الإلزامية

- إضافة فقط (Additive Only): يُفَضَّل إضافة أعمدة وجداول جديدة فقط دون حذف أو تعديل أعمدة موجودة لتقليل فقدان البيانات.
- NO DROP: يُمنع حذف أعمدة أو جداول في بيئة الإنتاج إلا بعد مرور فترة احتفاظ كافية (Deprecation Period).
- إعادة توليد الأنواع: بعد كل ترحيل، شغّل `supabase gen types` لتحديث `src/shared/supabase/types.ts`.
- التراجع (Rollback): كل ترحيل يجب أن يكون قابلاً للتراجع. اكتب ملف `.sql` عكسي للتراجع عند الحاجة.
- الاختبار: اختبر كل ترحيل على نسخة من قاعدة البيانات الإنتاجية قبل التطبيق الفعلي.
- التوثيق: أضف تعليقاً في أعلى كل ملف ترحيل يشرح الغرض والتأثير.

ترتيب عمليات الترحيل

- إنشاء ملف الترحيل الجديد عبر `supabase migration new {description}`
- كتابة SQL للتعديلات مع التزامن مع المخطط الحالي
- اختبار الملف محلياً عبر `supabase db reset`
- تشغيل `supabase gen types` لتحديث أنواع TypeScript
- مراجعة التغييرات على الأنواع وتحديث الكود المتأثر
- تطبيق الترحيل على بيئة Staging للاختبار
- النشر إلى الإنتاج عبر `supabase db push`
- التحقق من سلامة البيانات بعد الترحيل

عند إضافة أعمدة جديدة حساسة (مثل مفاتيح API أو بيانات شخصية)، يجب: استخدام تشفير `at-rest` عبر Vault أو التشفير التطبيقي، عدم تضمين البيانات الحساسة في نوع `TypeScript` المُؤلّد إذا أمكن، وإضافة RLS فورية لمنع الوصول غير المصرح به.

● تحذير الإنتاج

لا تُطبّق ترحيلات تحتوي على `DROP TABLE` أو `DROP COLUMN` أو `ALTER TYPE` مباشرة على بيئة الإنتاج. هذه العمليات قد تؤدي إلى فقدان بيانات أو توقف الخدمة. استخدم دائماً نهج `Additive Migrations` مع مرحلة إهمال (`Deprecation`) كافية.

19

استراتيجية النسخ الاحتياطي (Backup Strategy)

استراتيجية النسخ الاحتياطي في فيكسور تعتمد بشكل أساسي على ميزات `Supabase Cloud` المدمجة إضافية لتعزيز الحماية. النسخ الاحتياطي ليس خياراً بل ضرورة حتمية لحماية بيانات المستخدمين وضمان الخدمة. تتبع فيكسور نموذج 1-2-3 للنسخ الاحتياطي: ثلاث نسخ، على وسيطين مختلفين، مع نسخة واحدة خ

نسخ Supabase الاحتياطية المدمجة

يوفر `Supabase Cloud` نسخاً احتياطية تلقائية يومية لقاعدة البيانات مع الاحتفاظ بـ 7 أيام من النسخ في الخوادم. يتم تخزين النسخ على `Amazon S3` مع تشفير `AES-256`. يمكن استعادة قاعدة البيانات بالكامل إلى أي نقطة زمنية خلال فترة الاحتفاظ عبر لوحة تحكم `Supabase` أو عبر `API`.

الاستعادة لنقطة زمنية (Point-in-Time Recovery — PITR)

ميزة `PITR` في `Supabase` تسمح باستعادة قاعدة البيانات إلى أي لحظة محددة خلال آخر 7 أيام (أو أكثر حسب خطة الدفع). هذا ضروري في حالات: حذف بيانات بالخطأ، تنفيذ ترحيل معيب، أو اختراق أمني. يتم الاستعادة عبر لوحة التحكم. والوقت المستهدف وينشئ `Supabase` نسخة جديدة من قاعدة البيانات عند تلك النقطة. العملية تستغرق دقائق إلى ساعات حسب حجم البيانات.

النسخ الاحتياطي التطبيقي (Application-Level Backups)

- تصدير دوري: تصدير بيانات الجداول المرجعية (`premium_plans` ، `point_packs`) أسبوعياً بصيغة `JSON`
- نسخ الإعدادات: الاحتفاظ بنسخة من مخطط قاعدة البيانات (`schema.sql`) في المستودع مع كل ترحيل
- نسخ البيانات الحساسة: تصدير بيانات `user_memories` و `moxi_personas` بشكل منفصل لحمايتها
- مراقبة النسخ: إعداد تنبيهات عند فشل أي عملية نسخ احتياطي

سيناريوهات الاستعادة

السيناريو	الإجراء	الوقت المتأثر
حذف بيانات بالخطأ	PITR إلى اللحظة السابقة للحذف	5-30 دقيقة
ترحيل معيب	PITR + إصلاح الترحيل + إعادة التطبيق	30-60 دقيقة
فقدان قاعدة البيانات الكامل	استعادة من آخر نسخة يومية	1-4 ساعات
اختراق أمني	PITR + تدقيق الأمان + تحديث RLS	2-8 ساعات

اختبر عملية الاستعادة بشكل دوري (مرة شهرياً على الأقل) على بيئة منفصلة. لا تنتظر حاجة فعلية للاستعادة لمعرفة أن الاحتياطية تعمل. احتفظ بسجل لكل عملية نسخ واستعادة مع التاريخ والنتيجة.

نسخ Supabase الاحتياطية التلقائية لا تشمل ملفات Storage (الصور، التحليلات المخزنة). يجب إعداد نسخ احتياطي لـ Supabase Storage عبر `supabase storage CLI` أو مزود تخزين خارجي مثل `AWS S3`.

وثيقة رسمية لمنظومة فيكسور الهندسية

VIXOR — AI-Powered Trading Terminal